



PL-300

Search

CTRL

K



DAX – Datentypen, Funktionen, Operatoren und Variablen

Enders Training

CONTENTS

DAX-Datentypen

Der Sonderfall BLANK

DAX-Funktionen

Funktionen aus Excel

Funktionen, die nur in DAX existieren

DISTINCTCOUNT

DIVIDE

DAX-Operatoren

Arithmetische Operatoren

Vergleichsoperatoren

Textverkettung

Logische Operatoren

Operatorrangfolge

Implizite Typkonvertierung

DAX-Variablen

Vorteile von Variablen

Beispiel: Formel ohne und mit Variable

Zusammenfassung

Modul 3 · Skript 10

DAX – Datentypen, Funktionen, Operatoren und Variablen

Die Bausteine von DAX-Formeln im Detail

Aufbauend auf den Grundlagen aus Skript 09 — Berechnungstypen, Formelsyntax und Objektreferenzen — geht es in diesem Skript um die konkreten Bausteine, mit denen DAX-Formeln arbeiten: Welche Datentypen gibt es, welche Funktionen stehen zur Verfügung, wie verhalten sich Operatoren, und wie helfen Variablen dabei, Formeln lesbarer und schneller zu machen.

DAX-Datentypen

Jede Spalte in einem semantischen Modell hat einen Datentyp. Dieser steuert, welche Art von Werten gespeichert wird und wird in Power Query festgelegt. Bei berechneten Spalten leitet DAX den Datentyp automatisch aus der Formel ab. Measures haben ebenfalls Datentypen, die sich jedoch je nach Filterkontext zur Laufzeit ändern können.

Modelldatentypen und DAX-Datentypen sind verwandt, aber nicht immer identisch:

Modelldatentyp	DAX-Datentyp	Wertebereich / Hinweis
Ganze Zahl	64-Bit-Ganzzahl	-2^{63} bis $2^{63} - 1$
Dezimalzahl	Reelle 64-Bit-Zahl	ca. $-1,79 \times 10^{308}$ bis $1,79 \times 10^{308}$, max. 17 Dezimalstellen
Boolescher Wert	Boolescher Wert	TRUE oder FALSE
Text	Zeichenfolge	Unicode-Zeichenfolge
Datum	Datum/Uhrzeit	Alle Datumsangaben nach dem 1. Januar 1900
Währung	Währung	ca. $-9,22 \times 10^{14}$ bis $9,22 \times 10^{14}$, 4 Dezimalstellen mit fixer Genauigkeit
Nicht zutreffend	BLANK	Entspricht in manchen Fällen NULL in SQL

Der Sonderfall BLANK

BLANK DAX verwendet BLANK sowohl für den Datenbankwert NULL als auch für leere Zellen in Excel. BLANK bedeutet *nicht* Null (0) — es steht für das *Fehlen eines Werts*. Zwei DAX-Funktionen beziehen sich direkt darauf: **BLANK()** gibt BLANK zurück, **ISBLANK()** prüft, ob ein Ausdruck BLANK ergibt.

Wichtig BLANK kann zu unerwarteten Ergebnissen führen, insbesondere bei Vergleichen. Die meisten Vergleichsoperatoren behandeln BLANK wie 0, eine leere Zeichenfolge oder FALSE — der Operator **==** (genau gleich) ist die Ausnahme: er gibt nur TRUE zurück, wenn der Wert tatsächlich 0 ist, nicht wenn er BLANK ist. Bei unerwarteten Ergebnissen mit **IF()** und **ISBLANK()** auf BLANK prüfen.

DAX-Funktionen

Die DAX-Funktionsbibliothek enthält hunderte von Funktionen, die jeweils für ein bestimmtes Ziel entwickelt wurden. Da DAX aus dem Power Pivot-Add-In für Excel 2010 hervorgegangen ist, sind über 80 Funktionen direkt mit Excel kompatibel — ein bewusster Entscheidung, damit Excel-Anwender schnell produktiv mit DAX arbeiten können.

Funktionen aus Excel

Alle gängigen Excel-Aggregationsfunktionen stehen in DAX zur Verfügung: **SUM**, **COUNT**, **AVERAGE**, **MIN**, **MAX** und viele weitere. Der einzige Unterschied: In Excel übergibt man einen Zellbereich, in DAX einen Spaltenverweis.

Darüber hinaus sind viele Excel-Funktionen für Mathematik, Text, Datum/Uhrzeit, Informationen und Logik verfügbar, darunter: **ABS**, **ROUND**, **SQRT**, **LEN**, **LEFT**, **RIGHT**, **UPPER**, **DATE**, **YEAR**, **MONTH**, **NOW**, **ISNUMBER**, **TRUE**, **FALSE**, **AND**, **OR**, **NOT**, **IFERROR**.

Die **IF**-Funktion verhält sich wie in Excel — sie prüft eine Bedingung und gibt je nach Ergebnis einen von zwei Werten zurück:

```
IF(<logischer_test>, <wert_wenn_wahr>[, <wert_wenn_falsch>])
```

Wird kein Wert für den FALSE-Fall angegeben, gibt **IF** BLANK zurück.

Funktionen, die nur in DAX existieren

Viele Funktionen sind Power BI-spezifisch, da sie direkt mit der Datenmodellierung zusammenhängen:

- Funktionen zur **Beziehungsnavigation**
- Funktionen zur **Änderung des Filterkontexts**
- **Iteratorfunktionen**
- **Zeitintelligenzfunktionen**
- **Pfadfunktionen**

Zwei besonders häufig benötigte DAX-eigene Funktionen sind **DISTINCTCOUNT** und **DIVIDE**:

DISTINCTCOUNT

```
DISTINCTCOUNT(<Spalte>)
```

Zählt die Anzahl **unterschiedlicher** Werte in einer Spalte — d. h. jeder Wert wird nur einmal gezählt, unabhängig davon, wie oft er vorkommt. In einer Analyse ist die Anzahl *eindeutiger Kunden* (Kunden ohne Doppelzählung) oft wichtiger als die Gesamtzahl aller Kundeneinträge.

DIVIDE

```
DIVIDE(<Zähler>, <Nenner>[, <alternatives_ergebnis>])
```

Führt eine Division durch und behandelt Division durch Null automatisch — ohne dass ein expliziter Test des Nenners notwendig ist. Wenn kein alternatives Ergebnis angegeben wird und der Nenner Null oder BLANK ist, gibt **DIVIDE** BLANK zurück. Die Funktion ist auch leistungsmäßig besser optimiert als eine manuelle Prüfung mit **IF**.

Hinweis **DIVIDE** verwenden, wenn der Nenner ein Ausdruck ist, der Null oder BLANK zurückgeben könnte. Wenn der Nenner hingegen ein konstanter Wert ist (z. B. **/ 100**), ist der einfache Divisionsoperator **/** effizienter, da er keine zusätzliche Nullprüfung durchführt.

Hinweis DAX-Funktionen lassen sich am schnellsten über eine Websuche mit dem Schlüsselwort **DAX** gefolgt vom Funktionsnamen finden. Die vollständige Referenz ist unter *DAX-Funktionsreferenz* in der Microsoft-Dokumentation verfügbar.

DAX-Operatoren

DAX-Formeln können verschiedene Operortypen verwenden. Die meisten Operatoren und deren Rangfolge sind mit Excel identisch.

Arithmetische Operatoren

Operator	Operation
+	Addition
-	Subtraktion
*	Multiplikation
/	Division
^	Potenzierung

Vergleichsoperatoren

Vergleichsoperatoren vergleichen zwei Werte und geben TRUE oder FALSE zurück:

Operator	Bedeutung
=	Gleich
==	Genau gleich
>	Größer als
<	Kleiner als
>=	Größer oder gleich
<=	Kleiner oder gleich
<>	Ungleich

Wichtig Alle Vergleichsoperatoren — außer `==` — behandeln BLANK wie 0, eine leere Zeichenfolge oder FALSE. Das bedeutet: `[Revenue] = 0` ist TRUE, wenn Revenue entweder 0 oder BLANK ist. Dagegen ist `[Revenue] == 0` nur TRUE, wenn Revenue exakt 0 ist.

Textverkettung

Mit `&` werden zwei Textwerte zu einem einzigen zusammengefügt:

```
Model Color = 'Product'[Model] & "-" & 'Product'[Color]
```

Logische Operatoren

Operator	Bedeutung
<code>&&</code>	UND — TRUE nur wenn beide Ausdrücke TRUE sind
<code> </code>	ODER — TRUE wenn mindestens ein Ausdruck TRUE ist
<code>IN</code>	Logisches ODER gegen eine Werteliste — verwendet geschweifte Klammern als Tabellenkonstruktor
<code>NOT</code>	Kehrt einen booleschen Ausdruck um (FALSE → TRUE, TRUE → FALSE)

Beispiel Der `IN`-Operator filtert auf Australien und Neuseeland ohne mehrere ODER-Bedingungen:

```
ANZ Revenue =  
CALCULATE(  
    [Revenue],  
    Customer[Country-Region] IN {  
        "Australia",  
        "New Zealand"  
    }  
)
```

Operatorrangfolge

Wenn eine Formel mehrere Operatoren enthält, bestimmt die **Rangfolge** die Auswertungsreihenfolge (von oben = höchste Priorität):

Priorität	Operator
1	[^] Potenzierung
2	- Vorzeichen (z. B. –1)
3	* und / Multiplikation und Division
4	NOT
5	+ und - Addition und Subtraktion
6	& Textverkettung
7	=, ==, <, >, <=, >=, <> Vergleiche

Bei gleicher Priorität gilt Auswertung von links nach rechts. Mit **Klammern** lässt sich die Reihenfolge gezielt ändern.

Wichtig Rangfolgefehler sind eine häufige Fehlerquelle. Das folgende Beispiel liefert ein falsches Ergebnis, weil die Multiplikation vor der Subtraktion ausgeführt wird:

```
-- Falsch:
Extended Amount = Sales[Order Quantity] * Sales[Unit Price] * 1 - [Unit Price Discount Pct]

-- Richtig: Klammern erzwingen die Subtraktion zuerst
Extended Amount = Sales[Order Quantity] * Sales[Unit Price] * (1 - [Unit Price Discount Pct])
```

Formeln immer gründlich testen und bei unerwarteten Ergebnissen die Operatorrangfolge prüfen.

Implizite Typkonvertierung

DAX führt bei Bedarf automatisch **implizite Typkonvertierungen** durch, wenn Werte unterschiedlicher Datentypen mit Operatoren kombiniert werden — explizite Umwandlungen sind meist nicht nötig. Wenn jedoch ein Wert für die gewünschte Operation nicht kompatibel ist (z. B. ein Datum multiplizieren), gibt DAX einen Fehler zurück.

BLANK wird je nach Operator unterschiedlich behandelt:

- Arithmetische Operatoren: BLANK wird wie **0** behandelt
- Textverkettung: BLANK wird wie eine **leere Zeichenfolge** behandelt

DAX-Variablen

Variablen ermöglichen es, Zwischenergebnisse in einer Formel zu benennen und mehrfach zu verwenden. Sie werden mit dem Schlüsselwort **VAR** deklariert, und die abschließende **RETURN**-Klausel definiert das Ergebnis, das die Formel zurückgibt.

```
<Measurename> =  
VAR <Variablenname> = <Ausdruck>  
RETURN  
    <Ausdruck, der die Variable verwendet>
```

Vorteile von Variablen

- **Lesbarkeit** — komplexe Formeln werden durch sprechende Variablennamen verständlicher
- **Wartbarkeit** — eine Änderung an einem Ausdruck muss nur an einer Stelle vorgenommen werden
- **Leistung** — eine Variable wird nur einmal berechnet, auch wenn sie in der RETURN-Klausel mehrfach referenziert wird
- **Debugging** — zur Entwurfszeit kann die RETURN-Klausel temporär durch eine Variable ersetzt werden, um deren Wert zu prüfen

Beispiel: Formel ohne und mit Variable

Die folgende Formel berechnet das Umsatzwachstum im Jahresvergleich. Ohne Variable wird der Vorjahresumsatz **zweimal** berechnet:

```
-- Ohne Variable – Vorjahresumsatz wird zweimal ausgewertet:  
Revenue YoY % =  
DIVIDE(  
    [Revenue]  
    - CALCULATE(  
        [Revenue],  
        SAMEPERIODLASTYEAR('Date'[Date])  
    ),
```



```
CALCULATE(  
    [Revenue],  
    SAMEPERIODLASTYEAR('Date'[Date])  
)  
)
```

Mit einer Variable wird der Vorjahresumsatz **einmal** berechnet und gespeichert — die Formel ist schneller und leichter zu lesen:

```
-- Mit Variable – Vorjahresumsatz wird einmal berechnet:  
Revenue YoY % =  
VAR RevenuePriorYear =  
    CALCULATE(  
        [Revenue],  
        SAMEPERIODLASTYEAR('Date'[Date])  
    )  
RETURN  
    DIVIDE(  
        [Revenue] - RevenuePriorYear,  
        RevenuePriorYear  
    )
```

Beide Formeln liefern dasselbe Ergebnis — die Version mit Variable läuft aber in mindestens der Hälfte der Zeit, da `CALCULATE([Revenue], SAMEPERIODLASTYEAR(...))` nicht doppelt ausgewertet wird.

Zusammenfassung

Datentypen

7 Datentypen von Ganzzahl bis BLANK. BLANK ≠ Null — es bedeutet "kein Wert". Mit BLANK() erzeugen, mit ISBLANK() prüfen. Besondere Vorsicht bei Vergleichen mit BLANK.

Funktionsbibliothek

80+ Excel-kompatible Funktionen plus DAX-spezifische für Beziehungen, Filter, Iteratoren und Zeitintelligenz. DISTINCTCOUNT für eindeutige Werte, DIVIDE für sichere Division.

Operatoren

Arithmetisch (+, -, *, /, ^), Vergleich (=, ==, >, <, <>, ...), Text (&), Logisch (&&, ||, IN, NOT).

Rangfolge wie in Excel — Klammern nutzen um Reihenfolge zu steuern.

BLANK & Operatoren

= und <> behandeln BLANK wie 0. == (genau gleich) unterscheidet zwischen 0 und BLANK.

Arithmetisch wird BLANK als 0 behandelt, bei Textverkettung als leere Zeichenfolge.

Variablen

VAR deklariert, RETURN gibt das Ergebnis zurück. Werden nur einmal berechnet — verbessern Lesbarkeit, Wartbarkeit und Leistung. Unverzichtbar bei komplexen Formeln.

DIVIDE vs. /

DIVIDE verwenden wenn der Nenner Null oder BLANK sein kann — behandelt Division durch Null automatisch. Den Operator / verwenden wenn der Nenner eine Konstante ist — er ist dann schneller.

Modul 3 – Semantisches Modell & DAX



DAX Grundlagen –
Berechnungstypen, Syntax und
Formeln

Modul 3 – Semantisches Modell & DAX

DAX-Berechnungen erstellen

